

Android App Development Course Syllabus

Archer Infotech, Kothrud Pune | Classroom & live-online batches | Updated September 2026

Institute: Archer Infotech, Kothrud, Pune - IT training since 2009

Website: <https://archerinfotech.in>

Contact: +91 9850 678451 | info@archerinfotech.in

Mode: Classroom at Kothrud, Pune and live-online batches

Duration: 4 Months

Level: Intermediate

Course page: <https://archerinfotech.in/courses/mobile-app-development/android-development-training-in-pune>

Course Overview

Android is the dominant mobile platform in India by a wide margin — roughly 95%+ of Indian smartphones run Android, and Pune product engineering teams (Razorpay Pune, BharatPe Pune, Pine Labs Pune, Amagi for connected-TV, Whatfix, Drip Capital, plus the Pune-based Android teams of every major BFSI app) hire Android engineers continuously. Archer Infotech's Android Development training in Pune teaches the platform as it is actually built in 2026 — Kotlin (Google's preferred language since 2019, now fully default), Jetpack Compose for UI (the modern declarative replacement for XML layouts), Coroutines + Flow for async, MVVM with Hilt dependency injection, Room for local persistence, Retrofit for networking, plus the production tail (Firebase integration, ProGuard, Play Console publishing). Classroom in Kothrud, online live, and weekend batches available.

Why Learn Android in 2026

Android is the largest mobile-development hiring pool in Pune — Indeed Pune lists more than 700 active Android Developer / Mobile Developer (Android) openings as of May 2026. The biggest employers are Razorpay Pune, BharatPe Pune, Pine Labs Pune, Amagi (connected-TV Android apps), Whatfix Pune, Drip Capital, plus the Pune-based teams of major BFSI apps (HDFC, ICICI, Bajaj Finserv mobile, Kotak), e-commerce (Flipkart Pune teams), travel (MakeMyTrip Pune), plus the IT services majors with mobile-app practices (TCS, Infosys, Wipro, Cognizant, Capgemini).

What changed in 2026: Kotlin is the default; Java is legacy for new code (Google last updated their Java tutorials years ago). Jetpack Compose is now the default for new UI; XML layouts are 'reading legacy code' territory. Coroutines + Flow have replaced RxJava as the async pattern. The Single-Activity + Navigation Compose pattern has eclipsed multi-Activity / Fragment architectures. Hilt has eclipsed Dagger 2 for dependency injection (it's a Hilt-on-top-of-Dagger productivity wrapper). Room is the universal local-database choice. Modern targetSdk 34+ and the Edge-to-Edge Insets-aware UI patterns are baseline.

What this means for hiring: 2026 Pune Android JDs expect Kotlin fluency, Jetpack Compose at depth, Coroutines + Flow, MVVM with Hilt, Retrofit + Room, plus one published Play Store app (or a polished GitHub portfolio). Archer Infotech's curriculum is rebuilt around exactly these expectations — modern Android, Kotlin-first, Compose-first.

- 700+ active Android Developer roles on Indeed Pune (May 2026)
- Android = ~95% of Indian smartphones — by-far the largest mobile hiring pool
- Kotlin + Jetpack Compose + Coroutines + Hilt + Room — the 2026 stack
- Single-Activity + Navigation Compose — the modern architecture pattern
- Strong Pune fintech / consumer-tech / e-commerce hiring

Who This Course Is For

This course fits you if

- Engineering, BCS, MCA, or BSc-CS student targeting Android Developer roles
- Working web developer (any stack) wanting to switch to mobile
- Working Java developer wanting to add modern Android (Kotlin + Compose)
- Working AndroidJava developer wanting to migrate to Kotlin + Jetpack Compose
- Working iOS / Flutter developer wanting to add Android for cross-platform reach
- Career restarter targeting mobile-development as a high-demand re-entry path

This course is not the right fit if

- If you have no programming background — at least basic Java / Python / JavaScript / Kotlin is required
- If you cannot put in 8–10 hours per week of practice outside class
- If you only want a certificate sticker — Pune Android hiring screens hard on real Play Store apps or GitHub repos
- If your goal is iOS — pick our iOS / Swift course (or both for cross-platform reach)
- If you have 4+ years of production Android Compose experience — you'll be under-stretched

Detailed Curriculum

Module 1: Kotlin Language Foundations (Week 1)

Kotlin from a programmer-aware starting point, taught as a language in its own right before any Android API appears. Variables and types, null safety with `?.`, `?:` and the `!!` you learn to avoid, control flow, functions and default arguments, lambdas and higher-order functions, data classes, sealed classes and enums.

The idioms get real attention because they are what makes Kotlin code read as Kotlin: the scope functions (`let`, `run`, `apply`, `also`, `with`) and when each is the right one, extension functions, and delegation. Collections and the standard-library operators close the week.

- Variables, types and type inference
- Null safety — `?.`, `?:`, and why `!!` is a smell
- Functions, default arguments and named parameters
- Lambdas and higher-order functions
- Data classes, sealed classes and enums
- Scope functions — `let`, `run`, `apply`, `also`, `with`
- Extension functions and delegation
- Collections and standard-library operators
- Destructuring and smart casts

Module 2: Android Studio, Gradle & App Fundamentals (Week 2)

The platform itself. Android Studio setup, the SDK manager, emulator configuration and physical-device debugging over ADB. Gradle 8+ with the Kotlin DSL — modules, build types, product flavours, version catalogues and dependency management, taught properly because Gradle is where most beginners lose days.

Then the app model: the project and resource structure, qualifiers for density, locale and orientation, the manifest, Activity and Application lifecycle, configuration changes and process death — the last of which is the source of the bug every Android interviewer asks about.

- Android Studio, SDK manager and AVD setup
- ADB and physical-device debugging
- Gradle 8+ with the Kotlin DSL
- Build types, product flavours and version catalogues
- Project and resource structure with qualifiers
- The Android manifest and components
- Activity and Application lifecycle
- Configuration changes and process death
- Logcat, breakpoints and the debugger

Module 3: Jetpack Compose — Composables, State & Layout (Week 3)

The modern Android UI toolkit, and the mental model shift it demands. Composable functions, the declarative model, and recomposition — what triggers it, what skips it, and why an unstable parameter quietly makes a screen slow.

State is the centre of the module: `remember`, `rememberSaveable`, `mutableStateOf`, state hoisting, and the unidirectional data flow that keeps a Compose screen predictable. Layout follows with `Column`, `Row`, `Box`, the Modifier system and its order-sensitivity, plus `ConstraintLayout` for Compose and lazy lists.

- Composable functions and the declarative model
- Recomposition, skipping and stability
- `remember`, `rememberSaveable` and `mutableStateOf`
- State hoisting and unidirectional data flow
- `Column`, `Row`, `Box` and layout basics
- The Modifier system and why order matters
- `ConstraintLayout` for Compose
- `LazyColumn`, `LazyRow` and item keys
- Side effects — `LaunchedEffect`, `DisposableEffect`

Module 4: Material 3, Theming & Animation (Week 4)

Making an app look and feel like a 2026 Android app. The Material 3 component set — app bars, navigation bars, cards, dialogs, bottom sheets, chips and the rest — used as a design system rather than a widget catalogue.

Theming covers colour schemes, typography and shape, light and dark mode, and Material You dynamic colour drawn from the user's wallpaper. Animation closes the week: `animate*AsState`, `AnimatedVisibility`, `Crossfade`, transitions and gesture-driven motion, with accessibility and reduced-motion handling treated as a requirement rather than an extra.

- Material 3 component set in practice
- Colour schemes, typography and shape tokens
- Light, dark and Material You dynamic colour
- `animate*AsState` and `AnimatedVisibility`
- `Crossfade`, transitions and gesture-driven motion
- Custom drawing with Canvas
- Accessibility — content descriptions, touch targets
- Reduced-motion and font-scaling support

Module 5: Navigation & Multi-Screen Architecture (Week 5)

Moving between screens without losing state. Compose Navigation — the `NavHost` and `NavController`, route definitions, typed arguments, nested graphs and the back stack model that decides what a back press actually does.

Deep links and app links connect the app to the outside world, including the verification that makes an HTTPS link open your app rather than a browser. The module also covers passing results between destinations, bottom-navigation and drawer patterns, and adaptive layouts for tablets and foldables.

- `NavHost`, `NavController` and route definitions
- Typed arguments and safe navigation
- Nested graphs and the back stack
- Deep links and verified Android App Links
- Returning results between destinations
- Bottom navigation, tabs and navigation drawers
- Adaptive layouts for tablets and foldables
- Window size classes

Module 6: MVVM, ViewModels & Dependency Injection with Hilt (Week 6)

Production architecture. MVVM with AndroidX ViewModels — surviving configuration changes, `SavedStateHandle` for process death, and the UI-state modelling that keeps a screen's state in one immutable object rather than five loose flags.

Hilt supplies dependency injection on top of Dagger: modules, bindings, component scopes, `@HiltViewModel`, and qualifiers. The module closes on layered architecture — UI, domain and data — the repository pattern, and the use-case

layer, including an honest note on when that layer is worth its indirection.

- MVVM with AndroidX ViewModel
- SavedStateHandle and surviving process death
- Modelling UI state as one immutable object
- Hilt modules, bindings and scopes
- @HiltViewModel and constructor injection
- Qualifiers and multibindings
- Layered architecture — UI, domain, data
- The repository pattern
- Use cases and when the layer earns its keep

Module 7: Coroutines & Flow — Structured Concurrency (Week 7)

Asynchronous Android done correctly. Coroutines from first principles — suspend functions, launch and async, dispatchers and thread confinement, and structured concurrency with scopes, supervisorScope and cancellation that actually propagates.

Flow follows as the reactive stream type: cold flows, operators (map, filter, combine, debounce, flatMapLatest), StateFlow versus SharedFlow, stateIn and shareIn, and lifecycle-aware collection with repeatOnLifecycle — the idiom that stops a screen collecting in the background and draining a battery.

- Suspend functions, launch and async
- Dispatchers and thread confinement
- Structured concurrency, scopes and supervisorScope
- Cancellation and cooperative cancellation checks
- Exception handling in coroutines
- Flow basics and cold-stream semantics
- Operators — map, filter, combine, debounce, flatMapLatest
- StateFlow vs SharedFlow, stateIn and shareIn
- Lifecycle-aware collection with repeatOnLifecycle

Module 8: Networking — Retrofit, OkHttp & Serialization (Week 8)

Talking to a backend. Retrofit for typed REST clients, OkHttp underneath with interceptors for authentication, logging and retries, and connection and timeout tuning for the unreliable networks Indian users actually have.

Serialization uses kotlinx.serialization or Moshi, with the schema-mismatch handling that stops a backend change crashing the app. Error handling is modelled explicitly as a sealed result type covering network, HTTP and parsing failures, and the module ends on token refresh, certificate pinning and mocking the API for tests.

- Retrofit typed REST clients
- OkHttp interceptors — auth, logging, retry
- Timeouts and tuning for poor networks
- kotlinx.serialization and Moshi
- Handling schema mismatch safely
- Modelling errors as a sealed result type
- Token refresh and authenticators
- Certificate pinning
- Mocking the API with MockWebServer

Module 9: Local Persistence — Room & DataStore (Week 9)

Data that survives the app being killed. Room over SQLite — entities, DAOs, relations, type converters, and queries returning Flow so the UI updates automatically when the table changes. Migrations get proper time, including the destructive-migration shortcut and why shipping it costs users their data.

DataStore replaces SharedPreferences for key-value and typed preferences. The module closes on offline-first design: the database as single source of truth, cache invalidation, and a sync strategy that resolves conflicts deliberately rather than by

luck.

- Room entities, DAOs and relations
- Queries returning Flow for reactive UI
- Type converters and embedded types
- Migrations and the cost of destructive fallback
- Indices and query performance
- Preferences and Proto DataStore
- Offline-first with the database as source of truth
- Cache invalidation and sync strategy
- Paging 3 for large data sets

Module 10: Background Work, Services & Notifications (Week 10)

Work that outlives the screen, on a platform that has spent a decade restricting exactly that. WorkManager for deferrable guaranteed work — constraints, chaining, unique work, expedited jobs and retry with backoff.

Foreground services and the Android 14+ service-type declarations follow, along with Doze, App Standby and the battery-optimisation behaviour that makes background work fail on some OEM devices and not others. Notifications close the module: channels, the runtime POST_NOTIFICATIONS permission, and Firebase Cloud Messaging for push.

- WorkManager — constraints, chaining, unique work
- Expedited work, retry and backoff policy
- Foreground services and Android 14+ service types
- Doze, App Standby and OEM battery restrictions
- Notification channels and importance
- The POST_NOTIFICATIONS runtime permission
- Firebase Cloud Messaging for push
- Handling notification taps and deep links
- AlarmManager for exact timing, and its limits

Module 11: Firebase & Backend Integration (Week 10)

Firebase as the de-facto mobile backend for Pune fintech and consumer apps. Firebase Auth across email, Google and phone-number sign-in — the last being the dominant flow in the Indian market. Firestore for real-time data with security rules written and tested rather than left open.

Crashlytics for crash reporting with custom keys and non-fatal logging, Analytics for funnels, Remote Config for feature flags and staged rollouts, and App Distribution for pre-release builds. The module includes reading a real crash report and fixing the bug behind it.

- Firebase Auth — email, Google, phone number
- Firestore data modelling and real-time listeners
- Firestore security rules and testing them
- Cloud Storage for user-generated media
- Crashlytics — custom keys and non-fatal logging
- Reading a crash report and fixing the cause
- Analytics events and funnels
- Remote Config for feature flags
- Firebase App Distribution

Module 12: Testing — Unit, Compose UI & Instrumentation (Week 11)

Tests that catch regressions rather than merely raising the coverage number. JUnit with MockK for unit tests, Turbine for asserting on Flow emissions, and fakes preferred over mocks where a fake is simpler to read.

Compose UI testing covers the semantics tree, finders, assertions and synchronisation — including the idling problem that makes flaky tests. Instrumented tests with Espresso, Robolectric for JVM-speed Android tests, and screenshot testing close the module, along with test doubles for Room and Retrofit.

- JUnit and MockK for unit tests
- Turbine for testing Flow emissions
- Fakes versus mocks in practice
- Compose UI testing and the semantics tree
- Synchronisation, idling and flaky-test causes
- Espresso for instrumented end-to-end tests
- Robolectric for JVM-speed Android tests
- In-memory Room and MockWebServer test doubles
- Screenshot testing for UI regressions

Module 13: Performance, App Size & Security (Week 12)

The engineering that separates a shipped app from a good one. Macrobenchmark for startup and scroll performance, Baseline Profiles and Startup Profiles for faster cold starts, and the Compose-specific traps — unstable parameters, unnecessary recomposition, and `derivedStateOf`. Memory profiling and leak detection with LeakCanary.

App size is treated as a feature, since download size measurably affects install rates in India: R8 shrinking and obfuscation, resource shrinking, and App Bundle splits. Security closes the module — runtime permissions, scoped storage, EncryptedSharedPreferences and keeping secrets out of the APK.

- Macrobenchmark for startup and scrolling
- Baseline and Startup Profiles
- Compose performance — stability and `derivedStateOf`
- Memory profiling and LeakCanary
- R8 shrinking, obfuscation and keep rules
- Resource shrinking and App Bundle splits
- Runtime permissions and scoped storage
- EncryptedSharedPreferences and the Keystore
- Keeping API keys out of the APK

Module 14: Release Engineering & Play Store Publishing (Week 13)

Getting the app into users' hands and keeping it there. Signing configuration and Play App Signing, the Android App Bundle versus the legacy APK, and versioning strategy. The Play Console flow end to end: store listing, content rating, the Data Safety form, target-API requirements and the policy checks that reject a first submission.

Release tracks — internal, closed, open and production — with staged rollouts and halting a bad release. CI/CD with GitHub Actions builds, tests, signs and uploads a bundle automatically, and Play Vitals is read as the production feedback loop.

- Signing configuration and Play App Signing
- App Bundle vs APK and versioning strategy
- Play Console listing, rating and Data Safety form
- Target-API requirements and common policy rejections
- Internal, closed, open and production tracks
- Staged rollouts and halting a bad release
- GitHub Actions for build, test, sign and upload
- Play Vitals — ANRs and crash-rate thresholds
- In-app updates and in-app review

Module 15: Capstone Project & Placement Preparation (Weeks 14–15 + 1 week placement prep)

Full-time capstone work followed by structured interview preparation. You build and publish a complete Android application — Compose UI, MVVM with Hilt, a Room-backed offline-first data layer, Retrofit networking, WorkManager background sync, Firebase push, tests, and a signed bundle on an internal Play track.

Interview preparation matches how Pune Android panels actually run: a Kotlin and coroutines round, an Android-internals round covering lifecycle, process death and memory, and a UI round on Compose. Resume, LinkedIn and GitHub polish included.

- Capstone implementation, deployment and README
- Publishing to an internal Play Store track
- Kotlin and coroutines interview round
- Android internals round — lifecycle, process death, memory
- Jetpack Compose and UI-architecture round
- System design for mobile — offline, sync, pagination
- Resume and LinkedIn rewrite for Android JDs
- GitHub portfolio polish
- HR mock interview and salary negotiation

Module 16: AI-Assisted Development Workflow (Final Week)

The skill every 2026 hiring panel now probes for — building real work with AI in the loop, responsibly. Learn to drive AI assistants (GitHub Copilot, Claude, Cursor, and IDE-native AI) to scaffold and accelerate the tools and stack this course covers, generate tests, explain and refactor unfamiliar code, and cut the boilerplate — while keeping you firmly in control of every decision. Heavy focus on guardrails: reviewing each AI suggestion, spotting hallucinated APIs or wrong answers, and handling licensing and data-privacy concerns. Close with a mini-project that takes a deliverable end-to-end using an AI-assisted workflow, then fold the same tooling into version control and everyday team practice.

- AI assistants — GitHub Copilot, Claude, Cursor, IDE-native AI
- Effective prompting for this course's stack — scaffolding, boilerplate, config
- AI-assisted test generation and coverage
- Explaining, refactoring, and modernising unfamiliar code with AI
- AI-driven review, error detection, and quality checks
- Generating and maintaining documentation with AI
- AI debugging — interpreting errors, logs, and failing output
- Guardrails — reviewing output, avoiding hallucinations, licensing & data privacy
- Team workflow — AI in the editor, in reviews, and in delivery pipelines
- Mini-project — a deliverable built end-to-end with an AI-assisted workflow

Capstone Projects

Production Android App with Compose + MVVM + Room + Retrofit

A complete production-grade Android app — pick a real domain (expense tracker, recipe finder, fitness tracker, news reader, or BFSI-style finance dashboard). Jetpack Compose UI with Material 3, MVVM with Hilt, Coroutines + Flow, Retrofit for a real public API, Room for offline-first storage, Firebase Auth + FCM, plus full Compose UI tests + Macrobenchmark. Published to Play Store as an internal-test track (publishing fee covered by student — ~Rs. 2,500). Outcome: a public GitHub repository plus a Play Store listing — exactly what Pune Android hiring panels look at first.

- Technology: Kotlin + Jetpack Compose
- Technology: MVVM + Hilt + Coroutines + Flow
- Technology: Retrofit + Moshi
- Technology: Room + DataStore
- Technology: Firebase Auth + FCM
- Technology: Compose UI testing + Macrobenchmark
- Technology: Play Store internal-test track

Real-Time Collaboration / Chat App

A real-time application — Firebase Firestore or WebSocket-backed chat, presence, optimistic UI, offline message queue, push notifications via FCM, plus a small admin panel. Demonstrates the patterns Pune fintech / consumer-app teams hire on.

- Technology: Kotlin + Compose
- Technology: Firebase Firestore + FCM
- Technology: MVVM + Hilt
- Technology: Compose Navigation

- Technology: Image / file uploads

Offline-First BFSI / Fintech-Style App

An offline-first finance / payments-style app — local Room database as source of truth, sync queue for network operations, secure storage for tokens (EncryptedSharedPreferences / SQLCipher), biometric auth (BiometricPrompt), strict data-handling for PII / financial data. Demonstrates the patterns Pune BFSI mobile teams (HDFC / ICICI / Razorpay / BharatPe) test for at interview.

- Technology: Kotlin + Compose
- Technology: Room + SQLCipher for encrypted storage
- Technology: EncryptedSharedPreferences
- Technology: BiometricPrompt
- Technology: Sync queue patterns
- Technology: Network state handling

Career Outcomes

Android Developer is the largest mobile-development hiring pool in Pune in 2026 — Indeed Pune lists 700+ active openings, with continuous hiring at Razorpay Pune, BharatPe Pune, Pine Labs Pune, Amagi, Whatfix, plus the Pune-based teams of major BFSI apps (HDFC, ICICI, Bajaj Finserv, Kotak), e-commerce (Flipkart Pune), travel (MakeMyTrip Pune), plus the IT services majors. Compensation tracks Java / backend developers within $\pm 10\%$ at every band; senior Android engineers in BFSI / fintech often earn a small premium because demand outstrips supply.

What pulls an Android developer above the median band: Jetpack Compose fluency (the 2026 differentiator — many courses still teach XML), MVVM + Hilt + Coroutines architecture, one published Play Store app, plus testing discipline (Compose UI tests + Macrobenchmark). Our capstone projects are designed exactly around these signals.

Senior Android Engineer bands at the top end are reported as national figures (Pune-specific Indeed pages do not exist for these specific titles); Pune trends within $\pm 10\%$ of these figures.

Roles after this course

- Android Developer
- Mobile Developer (Android)
- Junior Mobile Engineer
- Software Engineer (Mobile)
- Cross-platform Mobile Developer (with Flutter / RN extension)

Indicative salary bands

- Android Developer (Pune) - Rs. 6,28,000 per year average (source: Indeed Pune (Android Developer))
- Junior Android Developer (Pune entry, <2 years) - Rs. 3,50,000 – Rs. 6,50,000 per year (source: AmbitionBox Pune Android Developer)
- Mid-level Android Developer (Pune, 3–5 years) - Rs. 10,00,000 – Rs. 17,00,000 per year (source: Glassdoor Pune Android Developer)
- Senior Android Developer (Pune, 5–8 years) - Rs. 16,00,000 – Rs. 28,00,000 per year (source: Glassdoor Pune Senior Android Developer)
- Lead Android Engineer (national, 8+ years) - Rs. 26,00,000 – Rs. 45,00,000 per year (source: 6figr India Lead Android Engineer (Pune $\pm 10\%$))

Companies hiring for these skills

- Razorpay (Pune)
- BharatPe (Pune)
- Pine Labs (Pune)
- Amagi
- Whatfix (Pune)
- Drip Capital
- HDFC Bank (Pune mobile teams)

- ICICI Bank (Pune mobile)
- Bajaj Finserv
- Kotak Mahindra Bank
- MakeMyTrip (Pune)
- Flipkart (Pune)
- TCS
- Infosys
- Cognizant
- Capgemini

Batch Modes and Duration

Duration: 11 weeks of structured curriculum plus 1 week of capstone and interview preparation (~3 months total). The original 4-month listing reflects optional extended evening format.

Classroom

- Location: Archer Infotech, Kothrud, Pune
- Morning batch — 10:00 to 13:00
- Evening batch — 18:00 to 21:00
- Lab access available outside class hours

Live online

- Same hours as classroom batches
- Recordings available for review
- Zoom for live sessions
- GitHub for code reviews
- Android Studio + emulator
- Slack / WhatsApp for async Q&A

Weekend

- Saturday + Sunday, 09:00 to 13:00
- Stretches over ~5 months instead of 3.

Maximum 15 students per batch.

Fees

Course fees range Rs. 20,000 – Rs. 90,000 depending on mode and concession. Play Console developer fee (~Rs. 2,500 one-time) is paid by the student.

Indicative range: Rs. 20,000 – Rs. 90,000

- Single payment with early-bird discount
- EMI in 2–3 instalments at no extra cost
- Corporate sponsorship — invoiced with GST

Placement Support

Placement support starts from week 9. By the end of the curriculum your resume highlights real published Android apps, your GitHub has at least two production-style repositories, and you have completed at least three mock technical interviews against question banks from Pune Android hiring teams.

We say placement support, not placement guarantee. Our support is unconditional, time-bound (six months after course completion), and includes free re-entry to a future batch's interview-prep sessions.

The placement process

- Week 9 — resume + LinkedIn rewrite for Android JDs

- Week 10 — GitHub portfolio cleanup, Play Store listings
- Weeks 11–12 — three rounds of mock technical interviews
- Week 12 — HR mock and salary negotiation coaching
- Post-course — referrals via our 17-year alumni network at 12+ partner companies (extra emphasis on Pune fintech / BFSI mobile)
- Up to 6 months of continued support
- Free re-entry to future batch interview-prep sessions

Partner companies

- Razorpay (Pune)
- BharatPe (Pune)
- Pine Labs (Pune)
- Amagi
- Whatfix (Pune)
- Bajaj Finserv
- TCS
- Infosys
- Cognizant
- Capgemini

Prerequisites and How to Start

Prerequisites: at least basic programming background (Java / Kotlin / Python / JavaScript / C# — any), basic OOP understanding, willingness to commit 8–10 hours per week of practice outside class. We expect basic programming fluency on day 1; we do not start from 'what is a variable'.

Suggested first steps

- Decide your mode — classroom, online live, or weekend
- Check the upcoming batch dates
- Book a free 30-minute counselling call
- Confirm enrolment and complete pre-course orientation (Android Studio install, emulator setup, Play Console signup guide)
- Show up to day one with a laptop running 64-bit OS, 16GB+ RAM (recommended for emulator), 50GB+ free disk

Frequently Asked Questions

How long does Android training in Pune take at Archer Infotech?

Approximately 3 months — 11 weeks of structured curriculum plus 1 week of capstone. Original 4-month listing is optional extended evening format. Weekend batch ~5 months.

What is the salary of an Android Developer in Pune?

Indeed Pune Rs. 6.28 lakh average (May 2026). Junior Rs. 3.5–6.5 lakh per AmbitionBox. Mid-level Rs. 10–17 lakh per Glassdoor. Senior Rs. 16–28 lakh. Lead Rs. 26–45 lakh nationally with Pune $\pm 10\%$.

Should I learn Java or Kotlin?

Kotlin — Google's preferred language since 2019, the default for new Android code. We cover Java as 'reading legacy code' only.

Jetpack Compose or XML layouts?

Jetpack Compose as default. XML covered as 'reading legacy code'. New Pune Android work in 2026 is overwhelmingly Compose.

Will I work on real projects?

Yes — three capstone projects with one published to Play Store internal-test track.

Native Android or Flutter / React Native?

Native Android for deepest fluency and largest Pune hiring pool. Flutter / RN for cross-platform reach.

Are weekend Android classes available in Pune?

Yes — Saturday and Sunday, 09:00–13:00, stretched over ~5 months instead of 3.

What is the fee?

Course fees range Rs. 20,000 – Rs. 90,000. Play Console fee (~Rs. 2,500) paid by the student.

What support do I get after course completion?

Six months of placement support, referrals via our alumni network at 12+ partner companies (extra emphasis on Pune fintech / BFSI mobile), resume / LinkedIn / GitHub rewrites.

Are the named trainers actually teaching?

Amol Chougule and Ankita Hartale personally lead every session of every batch.

Enquire or book a free demo session. Archer Infotech, Flat No. 12, Divyadarshan Housing Society, behind Kothrud Bus Stand Road, Kothrud, Pune 411038. Phone +91 9850 678451. Email info@archerinfotech.in. Web archerinfotech.in.